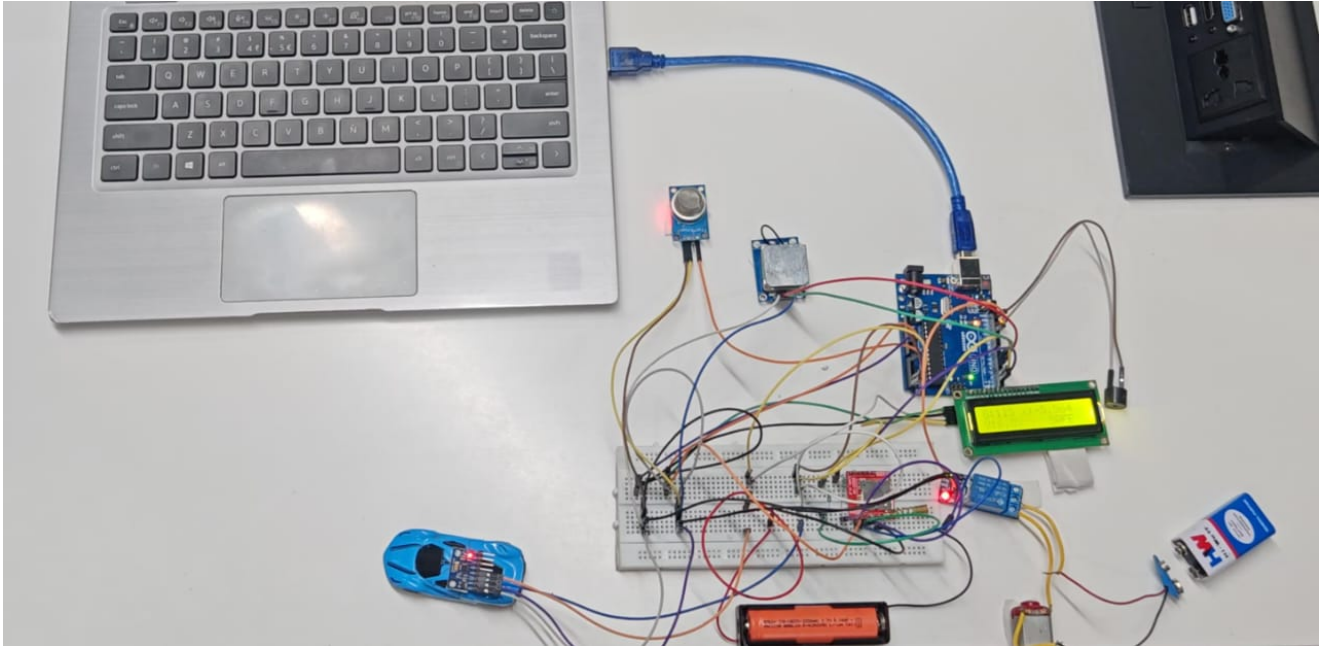


SMART ACCIDENT DETECTION & ALERT SYSTEM

Arduino-Based Vehicle Safety & Emergency Alert Prototype



Core idea: detect a possible vehicle accident, alert nearby people with a buzzer, show the emergency status on an LCD, and send an SMS through GSM communication.

Project status: Portfolio-ready reference package

Documentation note: The original Arduino sketch and final wiring record were not available. The implementation and pin plan in this document were reconstructed from the confirmed project components and behavior and should be verified before physical deployment.

01 — PROJECT SNAPSHOT

The Smart Accident Detection & Alert System is an embedded safety prototype built around an Arduino Uno. The system uses an MPU6050 motion sensor to identify a significant change in vehicle orientation. When the configured accident condition is detected, the controller activates a buzzer, displays **ACCIDENT DETECTED** on a 16×2 I2C LCD, and sends an emergency SMS through a SIM900L GSM module.

Category	Details
Domain	Embedded Systems / IoT / Vehicle Safety
Controller	Arduino Uno
Accident Sensor	MPU6050
Communication	SIM900L GSM
Display	16×2 I2C LCD
Additional Sensor	MQ135
Local Alert	Buzzer

02 — PROBLEM & SOLUTION

Problem: A vehicle accident can require immediate attention, but people nearby or an emergency contact may not know what happened immediately.

Solution: A compact embedded system monitors vehicle motion and provides a three-part response: **audible alert + on-device status + GSM SMS notification.**

03 — HOW IT WORKS

1	Monitor	MPU6050 continuously provides motion/orientation data.
2	Detect	A reference tilt threshold of about 45° is used to trigger the accident condition.
3	Alert	Buzzer turns ON and LCD displays ACCIDENT DETECTED.
4	Notify	SIM900L sends an emergency SMS to the configured number.

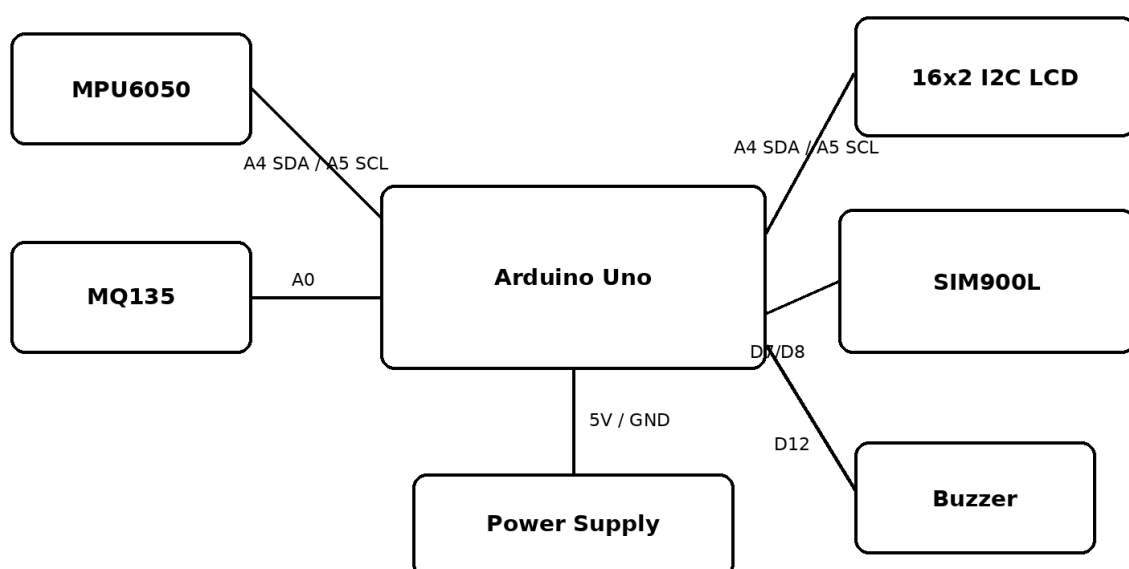
04 — HARDWARE & REFERENCE WIRING

Component	Role
Arduino Uno	Main controller and logic processing
MPU6050	Detects motion and vehicle tilt
SIM900L	GSM network communication and SMS
16×2 I2C LCD	Displays system and accident status
MQ135	Reads gas/air-quality level as supporting sensor data
Buzzer	Provides an immediate local warning
Power supply	Provides power to the prototype

Reference pin assignment used by the reconstructed sketch:

Module	Connection
MPU6050	SDA → A4, SCL → A5
16×2 I2C LCD	SDA → A4, SCL → A5
MQ135	Analog output → A0
Buzzer	→ D12
SIM900L	TX → D7; RX ← D8 (SoftwareSerial)

Smart Accident Detection & Alert System — Reference Wiring



Reference pin plan: MPU6050 + LCD share I2C (A4/A5); MQ135=A0; Buzzer=D12; SIM900L=D7/D8.
Verify SIM900L power and logic-level requirements before connecting to an Arduino Uno.

Reference circuit diagram — created for the reconstructed portfolio package, not recovered from the original project.

05 — FEATURES

- Accident/tilt detection using MPU6050
- Audible accident warning using a buzzer
- Real-time LCD accident status
- GSM emergency SMS through SIM900L
- MQ135 sensor reading for supporting environmental information
- Arduino-based embedded control

06 — SOFTWARE & LIBRARIES

Language: Arduino C/C++

Libraries: Wire, LiquidCrystal_I2C, SoftwareSerial, MPU6050

Controller environment: Arduino development workflow

07 — DETECTION LOGIC

The reference implementation calculates two orientation angles from MPU6050 accelerometer readings. If either angle reaches the configured 45° threshold, the accident state is triggered. The buzzer remains active while the accident state is present, the LCD changes to the emergency message, and the SMS routine is called once for that alert event.

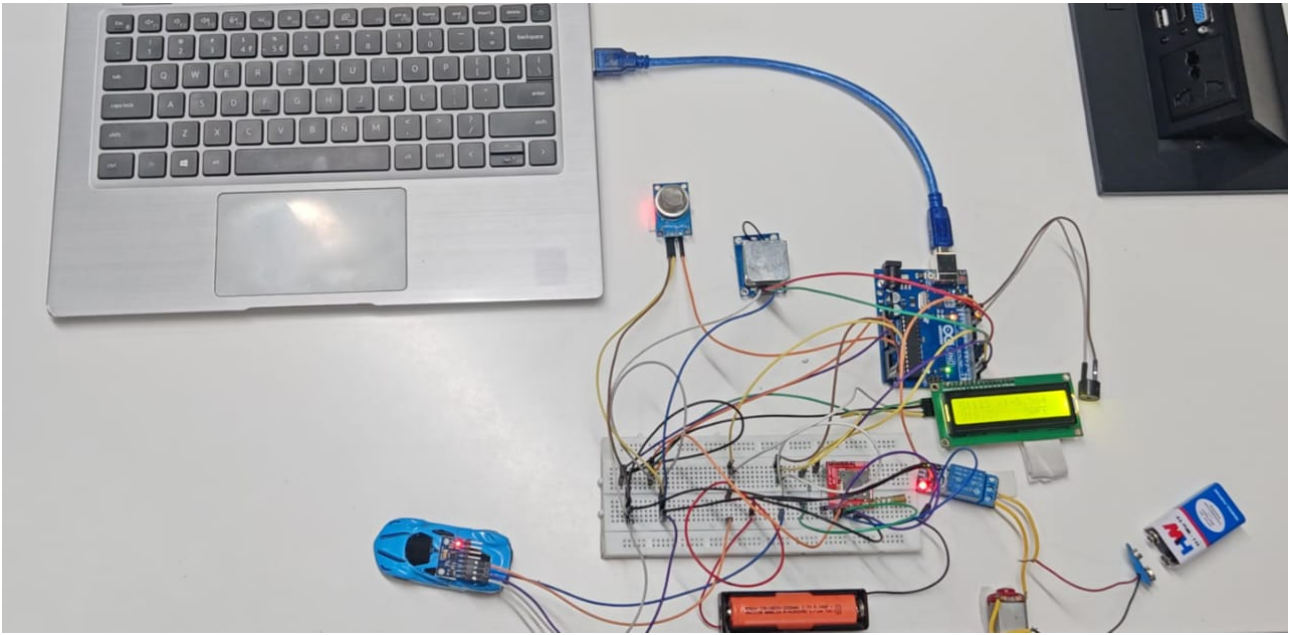
08 — EXPECTED USER EXPERIENCE

System Normal	Monitoring sensor values	Buzzer OFF
Accident Condition	LCD: ACCIDENT DETECTED	Buzzer ON
Notification	Emergency SMS	SIM900L

09 — FUTURE ENHANCEMENTS

GPS-based location sharing, manual emergency/cancel control, cloud event logging, a mobile/web dashboard, and more robust multi-sensor accident classification can extend the prototype.

10 — PROJECT PHOTOGRAPH



11 — COMPLETE ARDUINO CODE

The following is the combined reconstructed reference sketch supplied with this project package. Replace the placeholder emergency number before any real SMS test. Verify all wiring and power requirements.

```
001  /*
002   Smart Accident Detection & Alert System
003   -----
004   Reference/reconstructed implementation for the
005   project portfolio. Pin assignments were recreated
006   because the original sketch was not available.
007
008   Behavior:
009   Accident detected -> Buzzer ON
010                   -> LCD: "ACCIDENT DETECTED"
011                   -> SIM900L sends an SMS
012
013   Hardware:
014   Arduino Uno, MPU6050, 16x2 I2C LCD, SIM900L,
015   MQ135, Buzzer
016
017   IMPORTANT:
018   Verify wiring, GSM power supply, and sensor thresholds
019   before uploading to physical hardware.
020  */
021
022  #include <Wire.h>
023  #include <LiquidCrystal_I2C.h>
024  #include <SoftwareSerial.h>
025  #include <MPU6050.h>
026
027  LiquidCrystal_I2C lcd(0x27, 16, 2);
028  MPU6050 mpu;
029
030  // Arduino RX, TX for SoftwareSerial.
031  // Arduino D7 <- SIM900L TX
032  // Arduino D8 -> SIM900L RX (use appropriate level shifting)
033  SoftwareSerial sim900(7, 8);
034
035  const int BUZZER_PIN = 12;
036  const int MQ135_PIN = A0;
037
038  const int GAS_THRESHOLD = 400;
039  const float TILT_THRESHOLD = 45.0;
040
041  // Change this to the emergency phone number before real use.
042  const char EMERGENCY_NUMBER[] = "+91XXXXXXXXXX";
043
044  bool accidentAlertSent = false;
045
046  void setup() {
047    pinMode(BUZZER_PIN, OUTPUT);
048    digitalWrite(BUZZER_PIN, LOW);
049
050    Serial.begin(9600);
051    sim900.begin(9600);
052
053    Wire.begin();
054
055    lcd.init();
056    lcd.backlight();
057    lcd.clear();
058    lcd.setCursor(0, 0);
059    lcd.print("Smart Accident");
060    lcd.setCursor(0, 1);
061    lcd.print("Detection System");
062
063    mpu.initialize();
064
065    delay(2000);
066    lcd.clear();
067    lcd.print("System Ready");
068    delay(1000);
069  }
070
```

11 — COMPLETE ARDUINO CODE (continued)

```
071 void sendSMS() {
072     sim900.println("AT");
073     delay(1000);
074
075     sim900.println("AT+CMGF=1"); // SMS text mode
076     delay(1000);
077
078     sim900.print("AT+CMGS=\"");
079     sim900.print(EMERGENCY_NUMBER);
080     sim900.println("\");
081     delay(1000);
082
083     sim900.print("Emergency Alert: Accident detected.");
084     sim900.write(26); // CTRL+Z
085     delay(5000);
086 }
087
088 void showAccidentAlert() {
089     digitalWrite(BUZZER_PIN, HIGH);
090
091     lcd.clear();
092     lcd.setCursor(0, 0);
093     lcd.print("ACCIDENT");
094     lcd.setCursor(0, 1);
095     lcd.print("DETECTED");
096
097     if (!accidentAlertSent) {
098         sendSMS();
099         accidentAlertSent = true;
100     }
101 }
102
103 void loop() {
104     int16_t ax, ay, az;
105     int16_t gx, gy, gz;
106
107     mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
108
109     // Calculate tilt angle from accelerometer readings.
110     float angleX = atan2((float)ay, (float)az) * 180.0 / PI;
111     float angleY = atan2((float)ax, (float)az) * 180.0 / PI;
112
113     int gasValue = analogRead(MQ135_PIN);
114
115     bool accidentDetected =
116         (abs(angleX) >= TILT_THRESHOLD) ||
117         (abs(angleY) >= TILT_THRESHOLD);
118
119     if (accidentDetected) {
120         showAccidentAlert();
121     } else {
122         digitalWrite(BUZZER_PIN, LOW);
123
124         lcd.clear();
125         lcd.setCursor(0, 0);
126         lcd.print("System Normal");
127         lcd.setCursor(0, 1);
128         lcd.print("Gas:");
129         lcd.print(gasValue);
130
131         // Allows a new alert after the vehicle returns to normal.
132         accidentAlertSent = false;
133     }
134
135     delay(500);
136 }
```

12 — SAFETY & VERIFICATION NOTES

This document is a portfolio reconstruction, not a certification or a guarantee of real-world crash detection. The original sketch and final wiring were unavailable. Before powering the physical circuit, verify the Arduino pinout, SIM900L power supply/current capability, GSM antenna/network requirements, logic-level compatibility, sensor mounting, and the emergency phone number. Test in a controlled environment rather than in a moving vehicle.

13 — REPOSITORY CONTENTS

README.md — project overview and setup information

Arduino_Code/accident_detection.ino — complete reference Arduino sketch

Circuit_Diagram/circuit_diagram.png — reference wiring diagram

Images/project.jpg — original project photograph supplied for this package

Images/circuit.jpg — cropped wiring reference photograph

Documentation/Project_Report.pdf — project report

Documentation/Smart_Accident_Detection_Alert_System_Final.pdf — this polished combined report + code document